

Lab: Liveness vs Readiness Probes with NGINX Deployment

Lab Objectives

By the end of this lab, students will be able to:

- Understand why probes are required
 - Configure Liveness Probe
 - Configure Readiness Probe
 - Observe container restart vs traffic stop
 - Clearly differentiate Liveness vs Readiness behavior
-

Concept Overview

Why Health Probes?

Kubernetes needs a way to:

- Restart broken containers
- Stop traffic to unhealthy applications

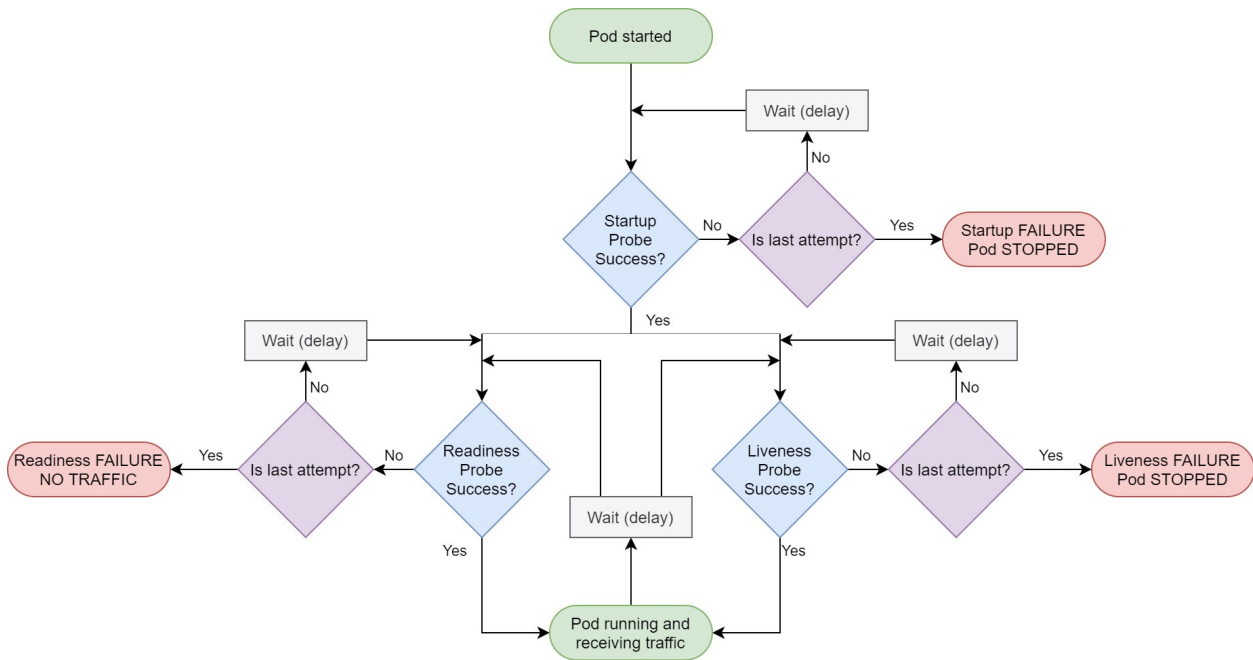
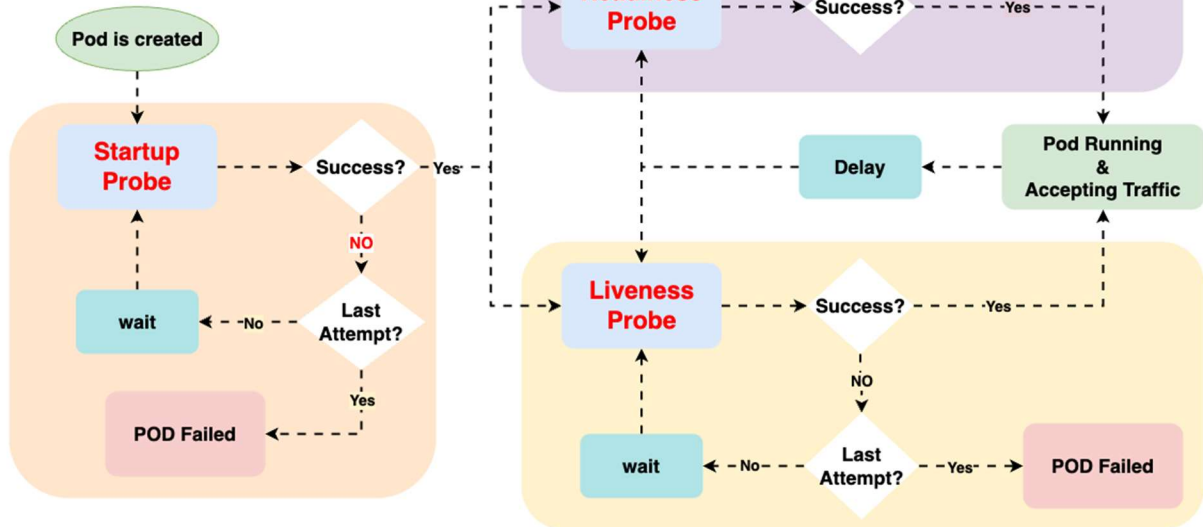
This is achieved using Probes.

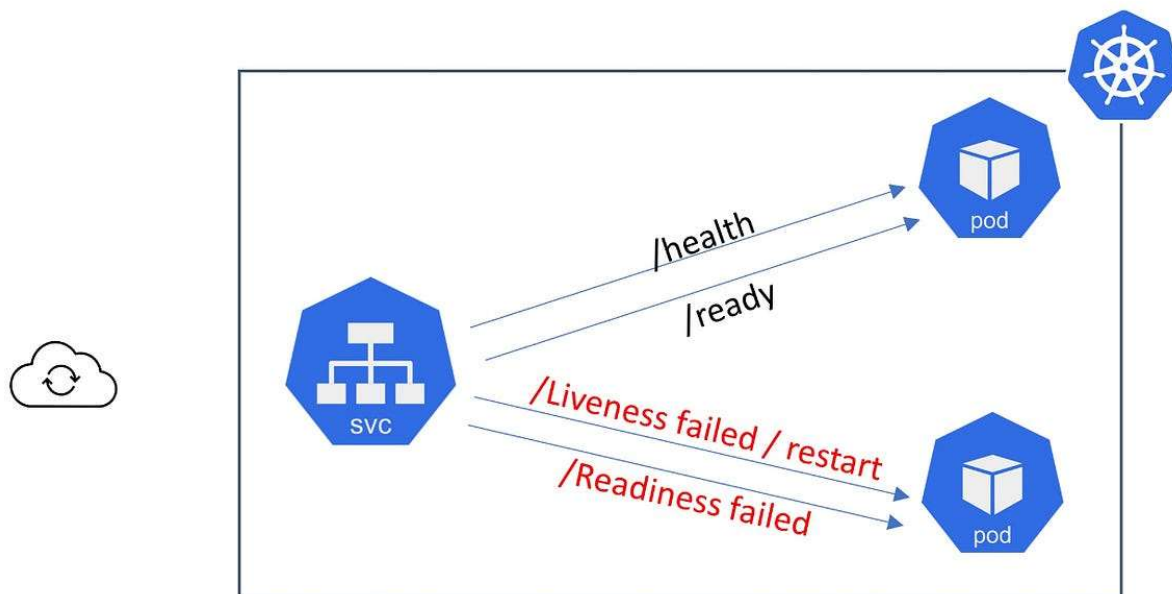
Probe Behavior – High Level Flow



Kubernetes Probes

Anvesh Muppada





✿ Types of Probes Used in This Lab

Probe Type	Purpose
Liveness Probe	Restart container if app is dead
Readiness Probe	Stop traffic if app is not ready

🔧 Lab Environment

Component Value

Cluster	Minikube / Any Kubernetes
App	NGINX
Service	ClusterIP
Probe Type	HTTP GET

Component Value

Port 80

Step 1: Create NGINX Deployment with Both Probes

Create file nginx-probes.yaml

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  name: nginx-probes
```

```
spec:
```

```
  replicas: 1
```

```
  selector:
```

```
    matchLabels:
```

```
      app: nginx
```

```
  template:
```

```
    metadata:
```

```
      labels:
```

```
        app: nginx
```

```
    spec:
```

```
      containers:
```

```
        - name: nginx
```

```
          image: nginx:latest
```

```
          ports:
```

```
            - containerPort: 80
```

livenessProbe:

httpGet:

path: /

port: 80

initialDelaySeconds: 10

periodSeconds: 5

failureThreshold: 3

readinessProbe:

httpGet:

path: /

port: 80

initialDelaySeconds: 5

periodSeconds: 5

failureThreshold: 3

Step 2: Create Service for Traffic Testing

Create nginx-svc.yaml

apiVersion: v1

kind: Service

metadata:

name: nginx-svc

spec:

selector:

app: nginx

ports:

- port: 80

targetPort: 80

type: NodePort

Apply:

```
kubectl apply -f nginx-probes.yaml
```

```
kubectl apply -f nginx-svc.yaml
```

Step 3: Verify Probes Configuration

```
kubectl describe pod <pod-name>
```

You should see:

Liveness: http-get http://:80/

Readiness: http-get http://:80/

Step 4: Test Readiness Probe (Traffic Stop)

Break Application Response (Soft Failure)

```
kubectl exec -it <pod-name> -- /bin/bash
```

```
mv /usr/share/nginx/html/index.html /tmp/
```

```
exit
```

Observe Pod Status

```
kubectl get pods
```

Output:

READY STATUS

0/1 Running

 Important

- Pod is Running
 - Container is NOT restarted
 - Traffic is stopped
-

Test Service Access

curl <NodeIP>:<NodePort>

Result:

Connection refused / 503

 Readiness probe removed pod from Service endpoints

 Step 5: Restore Readiness

```
kubectl exec -it <pod-name> -- /bin/bash
```

```
mv /tmp/index.html /usr/share/nginx/html/
```

```
exit
```

```
kubectl get pods
```

Output:

READY STATUS

1/1 Running

Traffic resumes 

 Step 6: Test Liveness Probe (Container Restart)

🔥 Kill NGINX Process (Hard Failure)

```
kubectl exec -it <pod-name> -- /bin/bash
```

```
nginx -s stop
```

```
exit
```

Watch Pod Events

```
kubectl get pods -w
```

You will see:

```
RESTARTS 1
```

Check events:

```
kubectl describe pod <pod-name>
```

Liveness probe failed

Killing container nginx

✅ Container restarted automatically

🔍 Side-by-Side Difference (MOST IMPORTANT)

Aspect	Liveness Probe	Readiness Probe
Purpose	Restart container	Stop traffic
Failure Result	Container restart	Pod removed from Service
Pod Status	Running	Running
Restart Count	Increases	No restart
Used for	Deadlock, crash	Startup delay, maintenance
Affects Service	❌ No	✅ Yes

Real-World Usage Mapping

Scenario	Correct Probe
App deadlock	Liveness
App slow startup	Readiness
DB down	Readiness
Infinite loop	Liveness
App warming cache	Readiness

Common Production Mistakes

- ✗ Using Liveness to check DB/API
 - ✗ Same delay values for both probes
 - ✗ Aggressive liveness timing
 - ✓ Liveness = Is app alive?
 - ✓ Readiness = Can app take traffic?
-

Lab Completion Checklist

- Deployment created
- Liveness probe tested (restart)
- Readiness probe tested (traffic stop)
- Service behavior observed
- Difference understood